ASSIGNMENT 8

OPTIMIZE ANYTHING

The 14-Day Vibe-Coding Challenge

Discover a process. Formulate the problem. Apply optimization. Prove the improvement.

Course	Optimization Techniques
Assignment duration	14 days (two weeks)
Development approach	Vibe coding / AI-assisted development is mandatory
Submission	Public GitHub repository and technical report

1. Assignment Overview

This is a two-week, hands-on optimization challenge—not a conventional large software project. Students will investigate how optimization can improve processes in any domain. They will identify inefficiencies, formulate objectives and constraints, select suitable methods from Chapter 6, use AI coding tools to build and test solutions, and support their conclusions with measurable evidence.

The central learning goal is to develop optimization as a general problem-solving skill. Algorithms are tools for improving a process; producing code alone is not sufficient. Students must explain what is being optimized, why the selected method is appropriate, and whether the result is genuinely better.

Assignment 8 is a compulsory ten-track challenge. Every student must complete all of Tracks A to J in Section 6: the Chapter 6 methods, a main case study of the student's choice, and compiler, xv6 operating-system, DBMS, quantum and software-engineering optimization. Each track is a focused experiment rather than a full project, and all work must be produced with documented AI-assisted (vibe) coding as set out in Section 8.

2. Learning Outcomes

- Recognize optimization opportunities in real or simulated processes.
- Translate a process into decision variables, an objective function, constraints, and measurable success criteria.
- Select and apply the methods covered in Chapter 6, explaining their suitability and limitations.
- Use vibe coding to generate, debug, refine, and test implementations while independently verifying correctness.
- Compare a baseline with optimized alternatives using fair, reproducible experiments.
- Interpret results, identify trade-offs, and explain when an optimization method should or should not be used.

3. Open-Domain Problem Selection

Students may choose any domain for the main case study (Track E). The examples below are suggestions only; they do not restrict the scope of the assignment. Tracks F to J fix their own domains (compilers, xv6, DBMS, quantum computing and software engineering), so the main case study is best drawn from a different area.

Domain	Possible optimization problems
Computer systems	OS scheduling, memory allocation, database query plans, compiler optimization
AI and machine learning	Model training, feature selection, hyperparameter tuning,

	inference efficiency
Industrial processes	Production scheduling, inventory, energy use, resource allocation
Quantum computing	QAOA, VQE, parameterized quantum circuits, hybrid quantum-classical optimization
Distributed computing	Load balancing, task scheduling, parallel execution, communication overhead
Transport and infrastructure	Routing, traffic management, timetabling, waste collection
Other domains	Healthcare, finance, cybersecurity, robotics, agriculture, education, personal productivity

4. Required Problem-Solving Process

1. **Discover the process:** Identify a process that consumes time, money, energy, memory, computation, or human effort.
2. **Establish the baseline:** Measure or simulate the current process. State the dataset, assumptions, hardware, and evaluation conditions.
3. **Formulate the problem:** Define decision variables, objective function, constraints, and the metric used to judge success.
4. **Select the method:** Explain why the chosen algorithm is suitable. Consider alternatives and state limitations.
5. **Use vibe coding:** Use AI tools to draft, modify, debug, and test code. Save prompts and revisions; do not treat generated code as automatically correct.
6. **Experiment and compare:** Compare the baseline and optimized approaches. Report objective value, runtime, resource use, quality, or other relevant measures.
7. **Analyze trade-offs:** Discuss accuracy versus speed, solution quality versus cost, scalability, overhead, and cases where the method may fail.

5. Mandatory Chapter 6 Coverage

Every algorithm and technique explicitly taught in Chapter 6 must be accounted for in the submission. Students must demonstrate a valid implementation, run an appropriate library-based experiment, or provide another verifiable experimental demonstration. Merely listing or describing a method is not sufficient.

Chapter 6 area	Required coverage
Foundations of optimization in ML	Machine learning as an optimization problem (parameters, loss functions, regularization), continuous versus discrete optimization, convex versus non-convex landscapes, and local minima, global minima and saddle points, each shown on a small example. Basic quantum computing concepts and the role of optimization in quantum machine learning.
Continuous optimization	Gradient Descent (GD), SGD and mini-batch SGD, Momentum, Nesterov Accelerated Gradient (NAG), AdaGrad, RMSProp, Adam, and the second-order or curvature-based methods covered in the course, including Newton and Gauss–Newton. Also cover backpropagation (distinct from the optimizer), curvature approximations (Fisher, diagonal or low-rank, Hessian-vector products), convergence for convex, strongly convex and non-convex cases, step-size and optimizer tuning, and noisy objectives.
Discrete optimization	Greedy methods, Integer Linear Programming (ILP), Branch-and-Bound, dynamic programming where covered, and feature selection or other combinatorial applications included in the course. Also cover QUBO and Ising formulations (binary-to-spin conversion and penalty terms for constraints) on problems such as Max-Cut, feature selection, scheduling or portfolio selection.
Quantum optimization	QAOA, VQE, quantum neural networks / parameterized quantum

	circuits, parameter-shift, SPSA, quantum natural gradient, and relevant hybrid quantum-classical methods covered in the course. Also cover every other quantum algorithm taught (for example Deutsch–Jozsa, Bernstein–Vazirani, Grover, Quantum Fourier Transform, Quantum Phase Estimation, Shor and quantum annealing) and the quantum machine learning methods taught (variational classifiers, quantum kernels). Study barren plateaus, finite-shot measurement cost and hardware noise, and the limits of embedding QUBO problems on annealing hardware.
Submodular optimization	Diminishing returns, testing whether the objective meets the relevant assumptions, greedy selection, and checking the conditions behind any claimed approximation guarantee.
Parallel program optimization	Load balancing, makespan, work stealing, locality, synchronization, communication overhead, and data, model, and pipeline parallelism. Measure speedup and parallel efficiency and interpret them with Amdahl's and Gustafson's laws.
Optimization strategy selection	Choose methods based on variable types, constraints, problem size, hardware, convergence requirements, and communication costs.
Distributed and multi-task optimization	FedAvg and other distributed or multi-task methods explicitly included in the Chapter 6 teaching material, such as MGDA and GradNorm where covered.

Important: not every method naturally fits the same real-world application. Use a portfolio of suitable experiments rather than forcing all algorithms into one application. Where a method is inapplicable to the main case study, demonstrate it on a smaller, appropriate benchmark and explain why it is unsuitable for the main problem. The final checklist must match the exact Chapter 6 syllabus taught in class.

6. Compulsory Experiment Tracks

All ten tracks (A to J) are compulsory. Each must state its objective, report a baseline, compare results under fair conditions, and include a correctness check. Tracks A to D cover the Chapter 6 methods in Section 5, Track E is the student-chosen main case study, and Tracks F to J are fixed domain tracks.

Track A — Continuous optimization. Compare the required continuous methods on suitable objective functions or an ML task. Examine convergence, final objective value, runtime, and sensitivity to settings. Include the second-order and curvature methods, backpropagation as the gradient computation behind a small ML model, and a convex and a non-convex objective (with a saddle point or local minima) to show how the landscape changes the result.

Track B — Discrete optimization. Use a selection, scheduling, allocation, or combinatorial problem. Compare exact methods and heuristics where feasible, covering greedy, ILP, Branch-and-Bound and dynamic programming. Formulate at least one problem as a QUBO/Ising model with penalty terms (for example Max-Cut or feature selection) and solve it classically as the baseline for Track C.

Track C — Quantum optimization. Use a simulator or available hardware for the quantum optimization methods in Section 5 (VQE, QAOA, PQCs and QNNs, parameter-shift, SPSA, quantum natural gradient). Compare with a classical baseline and distinguish simulation from real quantum hardware. Solve the Track B QUBO (such as Max-Cut) with QAOA, and demonstrate the effect of finite measurement shots, hardware noise and barren plateaus as circuit size grows.

Track D — Submodular, parallel and distributed optimization. Test greedy selection on an appropriate submodular objective. Investigate scheduling and parallelization (load balancing, work stealing, data, model and pipeline parallelism) and measure speedup, efficiency and overhead on multiple cores or processes. This track is where students show the advantage, and the limits, of parallel computing. Also demonstrate distributed and multi-task optimization: FedAvg against centralized training (varying local steps and communication rounds), and MGDA and GradNorm against equal task weighting on a two-task model.

Track E — Main real-process case study. Apply the Section 4 workflow to a domain chosen by the student and justify the selected methods with a short strategy-selection table (variable type, problem size, exact or heuristic, hardware, parallelism) and an experimental comparison of at least two methods. It may reuse the domain of Tracks F to J only if it goes substantially deeper.

Track F — Compiler optimization. Build a small compiler or optimization passes and show that they improve the generated code.

- **Build:** a mini-compiler for a toy language (lexer, parser, intermediate representation, code generator), or custom passes in LLVM, a GCC plugin or a similar framework.
- **Optimize:** implement at least two optimizations (four for full credit), for example constant folding and propagation, dead-code elimination, common sub-expression elimination, strength reduction, loop-invariant code motion, loop unrolling, inlining or graph-colouring register allocation.
- **Formulate:** pose at least one compiler decision (pass ordering, register allocation, instruction scheduling or unroll factor) as an optimization problem and solve it with a Chapter 6 method such as greedy, ILP or dynamic programming.
- **Measure and verify:** instruction count, run time, code size and compile time; optimized programs must produce the same output as unoptimized ones.

Track G — xv6 operating-system optimization. Modify the xv6 teaching operating system to learn OS concepts by optimizing them.

- **Change:** add or modify at least one kernel feature, for example a priority, lottery, stride or multilevel-feedback-queue scheduler, page replacement or copy-on-write fork, a better buffer cache, file-system allocation or disk scheduling, reduced lock contention, or system calls that report scheduling and memory statistics.
- **Scope:** one kernel feature benchmarked over at least three workloads is the minimum; a second feature earns full credit. Test after every change, because AI-generated kernel code often compiles but corrupts state.
- **Explain:** the OS concept involved (processes, scheduling, virtual memory, file system or synchronization) and the objective optimized, such as turnaround time, response time, throughput, fairness, page faults or disk I/O.
- **Measure and verify:** before-and-after results for the same workload under QEMU using new user-level benchmark programs; xv6 must still boot and pass usertests. Submit the patch or diff.

Track H — DBMS optimization. Optimize a database workload on a real DBMS (PostgreSQL, MySQL, SQLite or similar) or a small engine written by the student.

- **Optimize:** query rewriting and join-order selection (dynamic programming, greedy or ILP), index or materialized-view selection formulated as a knapsack or ILP problem, buffer-pool or cache replacement comparison, partitioning, or transaction scheduling.
- **Formulate:** state the chosen task (index selection, join order or view selection) as an optimization problem with variables, objective and a storage or time constraint, and compare at least two Chapter 6 methods (for example greedy against ILP or dynamic programming) with the default optimizer. Adding a single index is not enough.
- **Data:** an accepted benchmark or a documented synthetic dataset of stated size (for example TPC-H at scale factor 0.1).
- **Measure and verify:** compare EXPLAIN / EXPLAIN ANALYZE plans, latency, throughput and storage cost before and after; query results must be identical.

Track I — Quantum algorithms and QML advantage. Show what quantum algorithms and quantum machine learning offer compared with classical computing, and where they do not help.

- **Algorithms:** run every quantum algorithm taught in the course on a simulator (Qiskit, PennyLane, Cirq or similar), as listed in Section 5, and run at least one experiment on real hardware or a hardware noise model where access allows. Write the circuits for at least two algorithms yourself rather than only calling library implementations.
- **QML versus classical:** train a variational classifier, QNN or quantum kernel and an equivalent classical model on the same data; report accuracy, training time, parameter count and number of circuit evaluations.
- **Algorithmic advantage:** compare a quantum algorithm with a known complexity advantage (for example Grover versus linear search) with its classical counterpart using query or gate counts as well as wall-clock time.
- **Analysis:** explain when quantum approaches help and when they do not (simulation overhead, noise, barren plateaus, data loading), and contrast this with the parallel-computing speedups measured in Track D. An honest negative result, with correct experiments and a sound explanation, earns full credit.

Track J — Software-engineering optimization (AI, algorithms and other domains). Optimize a software-engineering task on a realistic codebase and show the effect with data.

- **Domain:** AI/ML (hyperparameter or architecture search, feature selection, training and inference speed, model compression, batching, token-cost reduction), algorithms (algorithm selection, data-structure tuning, cache-aware or parallel sorting, graph and dynamic-programming algorithms), or software engineering (test-suite minimization and prioritization, CI/build scheduling, profiling and refactoring, release planning, cloud-cost optimization). Other domains need instructor approval.
- **Formulate:** define decision variables, objective and constraints, and solve with at least one named Chapter 6 method compared with a naive baseline. A speedup without a formulation does not meet the track.
- **Scale:** use a non-trivial codebase, for example at least 1,000 lines or a public open-source project.
- **Measure and verify:** profile the baseline, identify the bottleneck, apply the optimization, and report the measured gain together with any cost in correctness, readability or maintainability.

7. Fourteen-Day Work Plan

Period	Focus	Expected work
Days 1–3	**Discover and formulate**	Select the main process, identify the inefficiency, obtain or generate data, formulate the objective and constraints, and measure the baseline.
Days 4–7	**Explore Chapter 6 methods**	Use vibe coding to implement or run the required methods on appropriate problems. Verify results and record errors and corrections.
Days 8–11	**Optimize and benchmark**	Tune settings, compare alternatives, measure performance, and investigate trade-offs.
Days 12–14	**Validate and demonstrate**	Finish experiments, prepare plots and tables, check reproducibility, complete prompt logs, and publish the repository and report.

Because all ten tracks are compulsory, keep each one small and focused. Suggested pacing: Days 1 to 3 select the Track E process, set up the environments (xv6 with QEMU, a compiler framework, a DBMS, a quantum simulator) and measure baselines; Days 4 to 7 complete Tracks A to D and the first optimizations for Tracks F to H; Days 8 to 11 complete Tracks E, I and J and benchmark; Days 12 to 14 validate, finish the prompt logs and publish.

8. Mandatory Vibe-Coding Practice

AI-assisted coding is compulsory. Students may use suitable AI coding assistants, but they remain responsible for the correctness, safety, and reproducibility of all submitted work.

Problem formulation: Analyze this process as an optimization problem. Identify decision variables, objective function, constraints, assumptions, and measurable metrics. Ask me for missing information instead of inventing it.

Algorithm selection: Compare the Chapter 6 methods that may suit this problem. Explain applicability, computational cost, assumptions, and why unsuitable methods should be rejected.

Implementation: Generate a small, testable implementation of the selected method. Include input validation, deterministic seeds where applicable, and comments explaining the optimization logic.

Verification: Create tests for edge cases and known examples. Independently verify the objective values and constraints; identify any assumptions that could make the result misleading.

Benchmarking: Design a fair comparison against the baseline. Keep data, stopping criteria, and evaluation conditions comparable, and report runtime and solution quality.

Critical review: Review the results skeptically. Identify bugs, unsupported claims, possible overfitting to the benchmark, overheads, and cases where the method may not improve the process.

Strict vibe-coding and AI-tool disclosure

Vibe coding is strict in this assignment: the substantial implementation in every track must be driven by prompts to an AI tool, and every use of AI must be disclosed. Any AI tool may be used, including Claude, ChatGPT, Gemini, GitHub Copilot, Cursor, Windsurf, Claude Code, Codex, DeepSeek, Perplexity, Grok and local models, but each tool used, including tools used only for debugging, explanation or documentation, must be named and covered in the submission.

- **AI tools register:** a table in the README listing every AI tool and model version used, the tracks (A to J) it was used for, the dates, and the kind of work (planning, code generation, debugging, testing, documentation).
- **Prompt logs:** a prompts/ directory with one sub-folder per track, holding the original prompt, the AI response or relevant excerpt, the revised prompt, the resulting code change (commit link), a verification note, and chat exports or screenshots from each tool. Remove API keys, passwords, tokens, private data and other secrets before publishing.
- **Commit traceability:** mark AI-assisted commits (for example the prefix [AI:tool-name]) and manual edits separately, so evaluators can see what was generated and what the student corrected.
- **Failure log:** at least three documented cases where AI-generated code was wrong, unsafe, inefficient or misleading (for example an optimizer that changed program behaviour, a kernel bug in xv6, an invalid SQL rewrite or an incorrect quantum circuit), with the fix and the lesson.
- **Tool comparison:** compare at least two AI tools on one task, noting correctness, hallucinations, speed and the number of prompt iterations needed.
- **Viva:** students must explain and modify their code live without the AI tool; components they cannot explain receive no credit.

Undeclared AI-generated content, or a prompt log that does not match the repository history, is academic misconduct.

9. Submission Requirements

- Public GitHub repository with a clear README, setup instructions, dependencies, and execution steps.
- Main process case study with problem statement, mathematical formulation, assumptions, and baseline.
- Chapter 6 coverage checklist mapping every taught algorithm or technique, including each quantum algorithm, to its experiment, code location, and result.
- Source code and test cases for all experiments, with external libraries and AI-generated components acknowledged.
- Baseline-versus-optimized results with tables, plots, measurement units, and reproducible settings.
- Prompt log documenting AI assistance, debugging, verification, and significant revisions, with the AI tools register, failure log and tool comparison required in Section 8.
- Technical report explaining outcomes, failures, limitations, trade-offs, and possible next steps.
- Short demonstration showing the problem, method, evidence of improvement, and key lessons.
- A report section for each of Tracks A to J, linked to its code and results (including the compiler passes, xv6 patch, DBMS query plans, and the QML and parallel-computing comparisons).

10. Academic and Technical Expectations

- Do not claim an improvement unless the measurements support it.
- Do not claim a theoretical guarantee unless the problem satisfies the guarantee's assumptions.
- Do not compare methods using unfairly different data, stopping criteria, or hardware conditions without explaining the difference.
- For stochastic algorithms, use repeated runs where feasible and report variability, not only the best run.
- A method that performs worse can still be a valuable result if the experiment is correct and the explanation is sound.
- Students must understand and be able to explain the submitted code, including AI-generated code.
- Do not claim a quantum or parallel speedup from timings that do not support it; state whether each result is theoretical, simulated, or measured on hardware.

11. Final Questions Students Must Answer

1. What process did you choose, and what evidence showed that it needed optimization?

2. How did you formulate the objective function and constraints?
3. Which Chapter 6 methods did you test, and why were they appropriate for their respective experiments?
4. How did the optimized solution compare with the baseline?
5. What trade-offs, failures, or limitations did you discover?
6. What evidence demonstrates that your results are correct and reproducible?
7. How could the optimization approach be adapted if the scale, data, constraints, or hardware changed?
8. For the compiler, xv6 and DBMS tracks, which optimization did you implement, what OS or database concept did you learn from it, and how did you show that correctness was preserved?
9. What advantages and limits of QML, quantum algorithms and parallel computing did your experiments reveal compared with classical computing?
10. Which AI tools did you use, how did their outputs differ, and which AI-generated errors did you find and fix?

Final Principle

The goal is not merely to produce an application. The goal is to learn how to recognize an optimization opportunity, formulate it correctly, choose and apply appropriate methods, use AI tools responsibly, and prove through experiments whether the process has improved.